МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ _Гуляев Александр Евгеньевич_____
(подпись) (фамилия, имя, отчество)

Группы _ИПо-42_____

Руководитель практической подготовки от профильной организации

_____IT-специалист_____
(должность)

_____ __Краснопёров Евгений Сергеевич_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _Берестов Дмитрий Борисович_____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....3
РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ.....5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии.....5
РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ.....8
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении.....8
РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ.....13
Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения.....13
Тема 3.2. Выполнение производственных заданий.....15
3.1.1. Использование выбранной системы контроля версий.....15
3.1.2. Разработка программного обеспечения.....15
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества.....23
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования.....24
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств.....26
3.1.8. Верификация и аттестация программного обеспечения.....29
ЗАКЛЮЧЕНИЕ.....32
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....33
ПРИЛОЖЕНИЯ.....35

ВВЕДЕНИЕ

Производственная практика является важной частью подготовки специалистов по специальности 09.02.07 «Информационные системы и программирование» и направлена на закрепление теоретических знаний, полученных в процессе обучения, а также на приобретение практических навыков профессиональной деятельности.

Практика имеет целью комплексное освоение студентами основных видов профессиональной деятельности по специальности, формирование общих и профессиональных компетенций, а также получение опыта практической работы в области разработки и сопровождения программного обеспечения и информационных систем.

Базой прохождения производственной практики стало Региональное отделение Всероссийской политической партии **«ЕДИНАЯ РОССИЯ»** Пермского края. В деятельности организации активно используются современные информационные технологии для ведения электронного документооборота, обработки информации, взаимодействия с гражданами и координации работы структурных подразделений. Использование программного обеспечения и информационных систем позволяет эффективно организовывать рабочие процессы и обеспечивать оперативный обмен данными между сотрудниками.

Производственная практика была направлена на освоение профессиональных компетенций в процессе выполнения практических задач, связанных с использованием информационных технологий, программного обеспечения и компьютерных систем в деятельности организации. В ходе практики проводилось изучение используемых программных средств, анализ информационных процессов, а также применение прикладного программного обеспечения для решения практических задач.

В настоящем отчёте представлен результат прохождения производственной практики студентом. Основными видами выполняемой деятельности являлись анализ требований к программным и

информационным системам, моделирование процессов разработки программного обеспечения, участие в разработке и доработке программных модулей, выполнение отладки программных компонентов, а также разработка тестовых наборов и тестовых сценариев для проверки работоспособности программного обеспечения.

Задачи производственной практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение.
2. Выполнять отладку программного модуля с использованием специализированных программных средств.
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
4. Производить инспектирование компонентов программного обеспечения на предмет соответствия стандартам кодирования.
5. Выбирать способы решения задач профессиональной деятельности применительно к различным условиям.
6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности.
7. Планировать и реализовывать собственное профессиональное и личностное развитие.
8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами и руководством.
9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей профессиональной деятельности.
10. Использовать информационные технологии в профессиональной деятельности.
11. Пользоваться профессиональной документацией на государственном и иностранном языках.

РАЗДЕЛ 1. ОРГАНИЗАЦИОННО-ПОДГОТОВИТЕЛЬНЫЙ ЭТАП ПРОХОЖДЕНИЯ ПРАКТИКИ НА ПРЕДПРИЯТИИ

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

Производственная практика проходила в Региональном отделении Всероссийской политической партии **«ЕДИНАЯ РОССИЯ»** Пермского края**,** которое является структурным подразделением политической партии и осуществляет свою деятельность на территории Пермского края в соответствии с Уставом партии и действующим законодательством Российской Федерации.

Организация расположена по адресу: 614039, г. Пермь, Комсомольский проспект, д. 72**.** Руководителем регионального отделения является Швецов Станислав Андреевич**,** который осуществляет общее руководство деятельностью организации.

Основными направлениями деятельности регионального отделения являются:

- координация деятельности местных и первичных отделений партии;
- организация и проведение общественно-политических мероприятий;
- взаимодействие с органами государственной власти и местного самоуправления;
- работа с гражданами и общественными организациями;
- развитие партийной структуры на территории региона.

В организационной структуре регионального отделения выделяются следующие основные подразделения:

- отдел организационной работы;
- отдел агитационно-пропагандистской работы;
- отдел партийного строительства;
- юридический отдел;
- бухгалтерия.

Каждое подразделение выполняет определённые функции, обеспечивая эффективную деятельность организации и реализацию поставленных задач.

В начале прохождения производственной практики был проведён инструктаж по правилам внутреннего распорядка, технике безопасности, охране труда и пожарной безопасности.

Общие функциональные обязанности работников организации

К основным функциональным обязанностям сотрудников относятся:

1. Соблюдение требований Устава партии, правил внутреннего трудового распорядка и локальных нормативных актов организации.
2. Соблюдение норм охраны труда, техники безопасности и производственной санитарии.
3. Обеспечение сохранности имущества, оборудования и служебной документации организации.
4. Поддержание порядка на рабочем месте и соблюдение санитарных норм в помещениях организации.
5. Исполнение распоряжений руководства в рамках своей должностной компетенции.
6. Соблюдение конфиденциальности информации, полученной в процессе выполнения служебных обязанностей.

Правила техники безопасности при работе с персональным компьютером

Сотрудники, работающие с компьютерной техникой, обязаны соблюдать следующие требования безопасности:

1. Монитор должен располагаться немного ниже уровня глаз (примерно на 5–10 градусов), при этом расстояние до экрана должно составлять 60–80 см.
2. Рабочее место должно быть оборудовано таким образом, чтобы освещение не создавало бликов на экране и не попадало напрямую в глаза пользователя.

3. Для предотвращения зрительного и физического утомления необходимо делать перерывы после каждого часа работы за компьютером.

4. Запрещается прикасаться к задней панели системного блока при включенном электропитании.

5. Не допускается подключение или отключение кабелей оборудования при включённом компьютере.

6. Запрещается размещать на системном блоке, мониторе или другой технике посторонние предметы.

7. Не допускается попадание жидкости на компьютерную технику и периферийные устройства.

8. Самостоятельное вскрытие, ремонт или модификация оборудования запрещаются.

Правила электробезопасности при работе с оргтехникой

При эксплуатации офисной техники необходимо соблюдать следующие правила:

1. Использовать только исправные электрические приборы, кабели и розетки.

2. Не оставлять включённое оборудование без присмотра.

3. Не допускать перегрузки электрической сети.

4. Не использовать самодельные удлинители и переходники.

5. В случае обнаружения неисправностей или повреждений оборудования необходимо немедленно сообщить ответственному сотруднику.

Соблюдение указанных требований позволяет обеспечить безопасные условия труда, предотвратить возникновение аварийных ситуаций и сохранить работоспособность используемой техники.

РАЗДЕЛ 2. ОЗНАКОМЛЕНИЕ СО СТРУКТУРОЙ И ХАРАКТЕРОМ ДЕЯТЕЛЬНОСТИ ПОДРАЗДЕЛЕНИЯ

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

Отдел организационной работы является ключевым структурным подразделением Регионального исполнительного комитета партии «ЕДИНАЯ РОССИЯ». Его деятельность направлена на обеспечение внутренней жизни партийной организации, координацию работы местных и первичных отделений, проведение партийных мероприятий и поддержку волонтерского движения.

Основные функции отдела организационной работы:

- Планирование и организация партийных мероприятий (съезды, конференции, собрания, форумы);
- Взаимодействие с местными и первичными отделениями, оказание им методической и организационной помощи;
- Ведение учета членов партии, активистов, волонтеров и сторонников;
- Подготовка и распространение партийной атрибутики, агитационных материалов;
- Обеспечение документооборота и делопроизводства в части организационной работы;
- Координация участия активистов в избирательных кампаниях;
- Взаимодействие с общественными организациями и объединениями.

В отделе работают специалисты по организационной работе, методисты, а также IT-специалист (и его помощник), отвечающий за техническое обеспечение перечисленных процессов.

Взаимодействие отдела с другими подразделениями осуществляется следующим образом: с отделом агитационно-пропагандистской работы — по вопросам информационного сопровождения мероприятий и организации агитационной деятельности; с юридическим отделом — по правовым вопросам проведения мероприятий, подготовки документов и работы с обращениями граждан; с бухгалтерией — по вопросам учета материальных

ценностей, финансового обеспечения и финансирования проводимых мероприятий.

Организационная структура Регионального отделения представлена на рисунке 1.

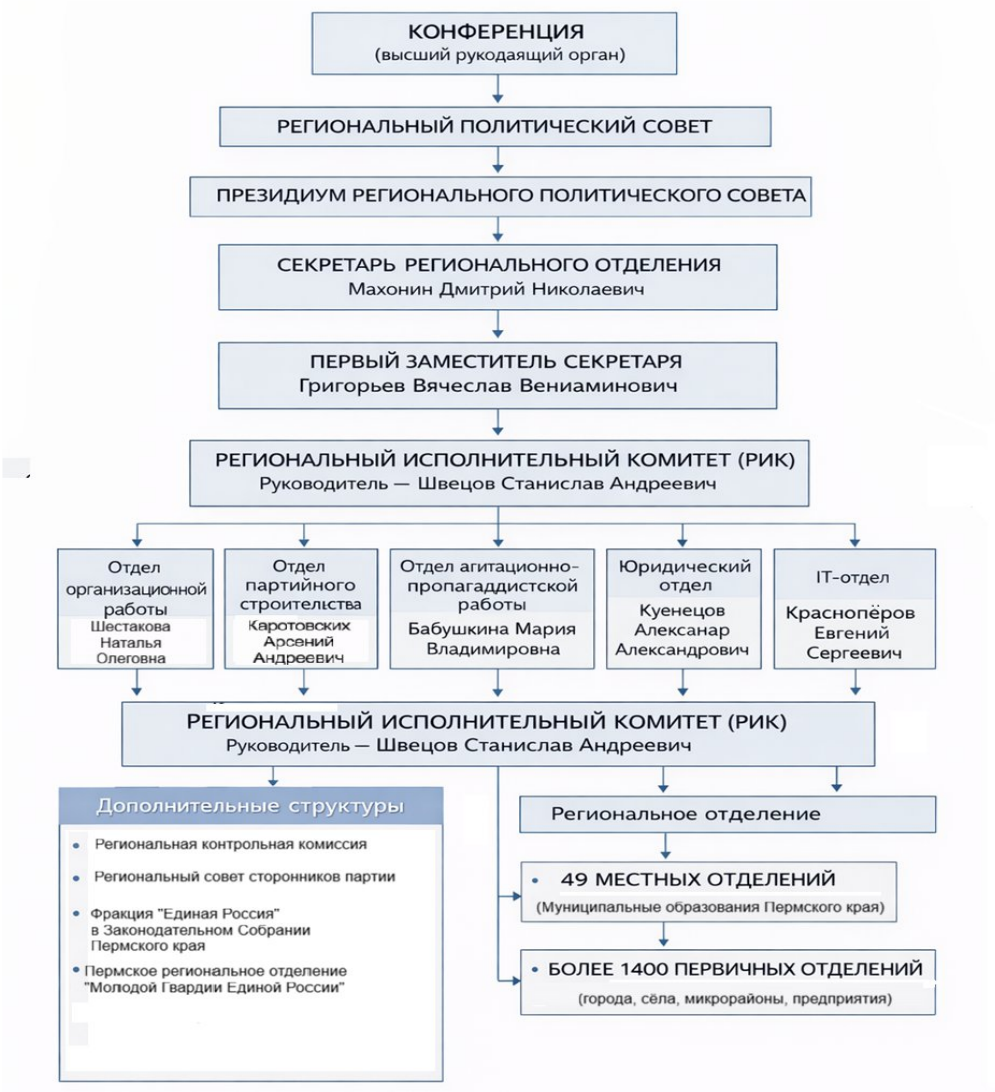

Рисунок 1 – Организационная структура Регионального отделения Всероссийской политической партии «Единая Россия» Пермского края

Региональное отделение Всероссийской политической партии **«ЕДИНАЯ РОССИЯ»** Пермского края представляет собой структурное подразделение политической партии, созданное для реализации уставных целей на территории субъекта Российской Федерации. Организационная структура Регионального отделения построена в соответствии с Уставом партии и Положением о Региональном отделении, утверждённым

Президиумом Регионального политического совета. Основными задачами Регионального отделения являются обеспечение партийного строительства, координация деятельности местных и первичных отделений, проведение избирательных кампаний, взаимодействие с органами государственной власти и местного самоуправления, а также работа с гражданами и общественными объединениями.

Структура Регионального отделения имеет иерархический характер и включает три уровня управления: стратегический (высшие руководящие органы), тактический (Региональный исполнительный комитет) и операционный (местные и первичные отделения). На рисунке 1 представлена схема организационной структуры Регионального отделения, отражающая состав подразделений и их функциональные взаимосвязи.

Высшие руководящие органы формируют стратегические направления деятельности партии. Высшим органом управления является Конференция регионального отделения, которая созывается не реже одного раза в два года. Конференция избирает Региональный политический совет (постоянно действующий коллегиальный орган) и Ревизионную комиссию (контрольно-ревизионный орган). Региональный политический совет определяет основные направления политической деятельности, утверждает бюджеты и программы, координирует работу местных отделений. Ревизионная комиссия осуществляет контроль за финансово-хозяйственной деятельностью и соблюдением уставных норм.

Оперативное управление повседневной деятельностью возложено на Региональный исполнительный комитет (РИК) во главе с руководителем — Швецовым Станиславом Андреевичем. РИК подотчётен Региональному политическому совету и является основным исполнительным органом, обеспечивающим реализацию решений вышестоящих партийных структур.

В структуру РИК входят функциональные отделы, каждый из которых выполняет специализированные задачи. Отдел организационной работы обеспечивает внутреннюю организационную деятельность: ведёт учёт членов

партии, активистов и волонтёров, координирует работу местных и первичных отделений, организует проведение партийных мероприятий, осуществляет документооборот и делопроизводство. Данный отдел играет ключевую роль в системе партийного строительства, поскольку обеспечивает методическое и организационное сопровождение деятельности нижестоящих структур.

Отдел агитационно-пропагандистской работы отвечает за информационное сопровождение деятельности партии: взаимодействие со средствами массовой информации, ведение официального сайта и аккаунтов в социальных сетях, разработку и распространение агитационных материалов, организацию публичных мероприятий, мониторинг общественного мнения и оперативное реагирование на информационные поводы.

Отдел партийного строительства курирует развитие партийной инфраструктуры на местах: оказывает поддержку местным и первичным отделениям, проводит обучение партийного актива, реализует партийные проекты, взаимодействует с общественными организациями и экспертным сообществом, привлекает новых сторонников.

Юридический отдел обеспечивает правовое сопровождение деятельности Регионального отделения: готовит и экспертирует договоры и соглашения, представляет интересы партии в судебных органах, консультирует сотрудников по правовым вопросам, контролирует соблюдение законодательства о политических партиях и выборах.

Бухгалтерия осуществляет финансовое планирование и учёт: ведёт бухгалтерский и налоговый учёт, готовит финансовую отчётность, рассчитывает заработную плату, учитывает материальные ценности и обеспечивает прозрачность финансовой деятельности Регионального отделения.

Режим работы организации:

Режим функционирования Регионального исполнительного комитета организован с учетом необходимости обеспечения непрерывного

взаимодействия с местными отделениями и оперативного реагирования на общественно-политические процессы. Для штатных сотрудников аппарата установлена пятидневная рабочая неделя с двумя выходными днями (суббота и воскресенье). Рабочий день продолжается с 9:00 до 18:00, с перерывом на отдых и питание продолжительностью один час (с 13:00 до 14:00), что соответствует требованиям трудового законодательства Российской Федерации.

Специфика деятельности политической организации обусловливает особенности трудового распорядка. В периоды активных избирательных кампаний (агитационный период, дни голосования), а также при подготовке и проведении крупных партийных мероприятий (съезды, форумы, встречи с избирателями) для сотрудников Регионального исполнительного комитета может устанавливаться особый режим работы. Он характеризуется возможностью привлечения к выполнению служебных обязанностей в выходные и праздничные дни, а также применением ненормированного рабочего дня. Данные меры обусловлены необходимостью обеспечения эффективности организационной деятельности и присутствия сотрудников на ключевых мероприятиях.

В период прохождения практики студент выполнял обязанности помощника IT-специалиста отдела организационной работы. В данной должности он подчинялся непосредственно IT-специалисту и руководителю отдела, осуществляя задачи по техническому сопровождению информационной инфраструктуры организации.

РАЗДЕЛ 3. РАБОТА НА РАБОЧИХ МЕСТАХ ИЛИ В ПОДРАЗДЕЛЕНИЯХ ПРЕДПРИЯТИЯ

Тема 3.1. Ознакомление с организацией информационного обеспечения подразделения

В период прохождения производственной практики было проведено ознакомление с организацией информационного обеспечения подразделения Регионального отделения Всероссийской политической партии **«ЕДИНАЯ РОССИЯ»** Пермского края. Информационное обеспечение подразделения представляет собой совокупность программных средств, баз данных, информационных систем и технических ресурсов, используемых для обработки, хранения и передачи информации в процессе деятельности организации.

В работе подразделения используются различные информационные системы, обеспечивающие эффективную организацию документооборота и взаимодействие сотрудников. Одной из основных систем является система электронного документооборота «1С: Документооборот», предназначенная для регистрации, хранения и обработки служебных документов. Для внутреннего взаимодействия сотрудников и организации рабочих процессов применяется корпоративный портал Bitrix24, который позволяет вести задачи, обмениваться информацией и контролировать выполнение поручений.

Для хранения и обработки информации об участниках и активистах партии используется специализированная база данных «Актив», функционирующая на основе системы управления базами данных PostgreSQL. Данная база данных предназначена для ведения учета участников, хранения контактной информации и формирования отчетности.

В процессе работы также используются системы для обработки обращений граждан, а также официальный сайт организации, разработанный

на платформе 1С-Битрикс, который обеспечивает информирование населения о деятельности партии и проводимых мероприятиях.

Рабочие места сотрудников оснащены персональными компьютерами с установленной операционной системой Windows 10. Для выполнения служебных задач используются стандартные офисные программные продукты, такие как Microsoft Office, а также веб-браузеры для работы с интернет-ресурсами и внутренними информационными системами организации. Для обмена электронной корреспонденцией используется корпоративная почтовая система Яндекс 360.

Организация информационного обеспечения подразделения включает процессы сбора, обработки, хранения и передачи информации. Документы регистрируются в системе электронного документооборота, после чего распределяются между ответственными сотрудниками. Информация о мероприятиях и деятельности подразделения размещается во внутренних информационных системах и на официальных интернет-ресурсах организации.

Таким образом, информационное обеспечение подразделения позволяет эффективно организовать работу сотрудников, обеспечить хранение и обработку служебной информации, а также повысить эффективность взаимодействия между подразделениями организации.

При проектировании новой информационной системы сотрудники отдела разработки и внедрения программных решений придерживаются каскадной модели жизненного цикла программного обеспечения (рисунок 2).

Рисунок 2 – Каскадная модель разработки программного обеспечения

Тема 3.2. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

Для сохранения, управления и дальнейшего распространения создаваемой программы используется локальная система контроля версий Git совместно с облачным сервисом GitHub**.** Эти инструменты позволяют эффективно вести исходный код, отслеживать историю изменений, создавать резервные копии и организовывать совместную работу над проектом.

Система Git работает на компьютере разработчика и представляет каждый проект в виде отдельного репозитория. В нём фиксируются все внесённые изменения. При этом сохраняется не только факт изменения, но и содержимое файлов на каждом этапе (коммит). Это обеспечивает возможность при необходимости откатить нежелательные изменения.

Кроме того, Git поддерживает создание веток, что позволяет одновременно развивать разные направления проекта, тестировать новые идеи или разделять стабильную и рабочую версии программы.

Для обмена репозиториями и совместной работы используются облачные сервисы, такие как GitHub**.** Этот сервис не только хранит репозитории, но и позволяет организовать командную работу, применять методологии CI/CD, проводить автоматическое тестирование и публиковать приложения на сервер организации. Также пользователи могут оставлять отзывы и сообщать об обнаруженных ошибках, что способствует улучшению качества программного продукта.

3.1.2. Разработка программного обеспечения

Перед началом разработки программного обеспечения исполнители выбирают подходящую методологию, которая позволяет систематизировать процесс работы, определить этапы разработки и обеспечить контроль за выполнением задач. Существует большое количество методологий разработки, однако для данного проекта была выбрана каскадная (водопадная) методология.

Каскадная методология предполагает последовательное выполнение пяти ключевых стадий:

1. Сбор требований – анализ и формулирование требований к разрабатываемой программе;
2. Проектирование – разработка архитектуры ПО, определение логики работы программы, выбор средств разработки, формирование конечного вида продукта;
3. Разработка – непосредственное создание программного кода и реализация функционала;
4. Тестирование и отладка – проверка работоспособности отдельных компонентов и всего ПО в целом, выявление и исправление ошибок;
5. Внедрение и сопровождение – эксплуатация продукта, сопровождение и исправление выявленных проблем во время работы.

Главные особенности каскадной методологии заключаются в следующем:

- фиксированные сроки и стоимость разработки;
- чётко определённые требования к продукту;
- отсутствие возврата к предыдущим этапам после их завершения.

Результатом применения этой методологии является полностью протестированное программное обеспечение с актуальной документацией.

Для данного проекта выбор каскадной методологии обусловлен рядом факторов:

1. Все требования к программному модулю заранее определены (см. приложение 1), а сроки выполнения работы ограничены;
2. Простота методологии делает её подходящей для относительно небольших проектов.

Разработка программного модуля будет осуществляться в интегрированной среде разработки Visual Studio. Эта среда предоставляет широкий набор инструментов для написания, компиляции, отладки и запуска

кода, а также включает средства для тестирования, публикации и визуального проектирования интерфейсов приложений. Visual Studio поддерживает множество языков программирования и позволяет использовать расширения, которые упрощают процесс разработки и добавляют новые возможности. Кроме того, среда включает готовые шаблоны проектов и инструменты для работы с различными платформами и технологиями.

Для хранения и распространения исходного кода проекта будут использоваться локальная система контроля версий Git и облачный сервис GitHub, что обеспечивает резервирование данных, отслеживание изменений и организацию командной работы.

Перед созданием приложения важно определить архитектуру данных, которая определяет структуру моделей данных, их содержимое, способы хранения и взаимодействие с клиентской частью программы. Это позволяет выстроить логичную и эффективную работу приложения с информацией на всех уровнях.

Описание основных сущностей данных (рисунок 3) представляет собой следующее:

1. Партнер:
   - уникальный идентификатор,
   - название,
   - идентификатор типа партнера,
   - рейтинг,
   - адрес,
   - ФИО директора,
   - номер телефона,
   - электронная почта;
2. Тип партнера:
   - уникальный идентификатор,
   - название;

3. Продукт:

- уникальный идентификатор,
- артикул,
- название;

4. История реализации продукции:

- уникальный идентификатор,
- идентификатор партнера,
- идентификатор продукта,
- количество реализованной продукции,
- дата продажи.

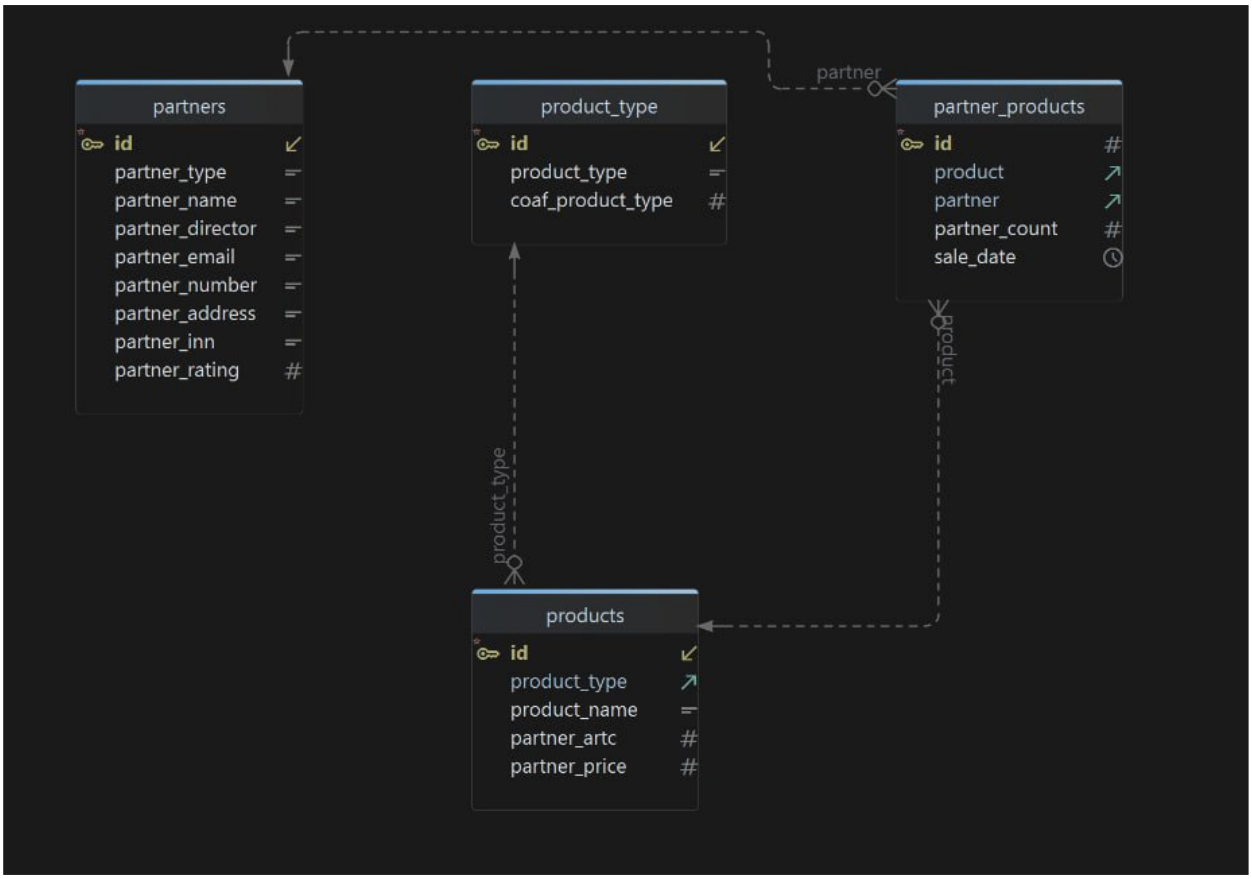

Рисунок 3 – Диаграмма основных сущностей

Пример кодового представления одной из данных сущностей представлен на рисунке 4.

```
GulyaevLib                                   GulyaevLib.ProductType
using System;
using System.Collections.Generic;

namespace GulyaevLib;

Ссылок: 3
public partial class ProductType
{
    Ссылок: 2
    public int Id { get; set; }

    Ссылок: 1
    public string? ProductType1 { get; set; }

    Ссылок: 1
    public float? CoafProductType { get; set; }

    Ссылок: 1
    public virtual ICollection<Product> Products { get; set; } = new List<Product>();
}
```

Рисунок 4 Код класса «product_type»

Для взаимодействия приложения с базой данных определяется контекст базы данных (рисунок 5) и программные сервисы (рисунок 6) для выполнения операций над данными в базе данных.

```
GulyaevLib                                   GulyaevLib.ApplicationContext
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace GulyaevLib;

Ссылок: 22
public partial class ApplicationContext : DbContext
{
    Ссылок: 4
    public ApplicationContext()
    {
    }

    Ссылок: 0
    public ApplicationContext(DbContextOptions<ApplicationContext> options)
        : base(options)
    {
    }

    Ссылок: 0
    public virtual DbSet<MaterialType> MaterialTypes { get; set; }

    Ссылок: 18
    public virtual DbSet<Partner> Partners { get; set; }

    Ссылок: 13
    public virtual DbSet<PartnerProduct> PartnerProducts { get; set; }

    Ссылок: 10
    public virtual DbSet<Product> Products { get; set; }

    Ссылок: 0
    public virtual DbSet<ProductType> ProductTypes { get; set; }

    Ссылок: 0
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        => optionsBuilder.UseNpgsql("HOST=localhost;DATABASE=gulyaev_db;USERNAME=app;PASSWORD=123456789");

    Ссылок: 0
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<MaterialType>(entity =>
        {
            entity.HasKey(e => e.Id).HasName("material_type_pkey");

            entity.ToTable("material_type", "app");

            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.CoafBreak).HasColumnName("coaf_break");
            entity.Property(e => e.MaterialType1)
                .HasMaxLength(50)
                .HasColumnName("material_type");
        });

        modelBuilder.Entity<Partner>(entity =>
        {
            entity.HasKey(e => e.Id).HasName("partners_import_pkey");
75 %    Проблемы не найдены.
Список ошибок   Вывод
Готово
```

Рисунок 5 – Часть контекста базы данных

```
using System;
using System.Collections.Generic;
using System.ComponentModel.Design;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;

namespace GulyaevLib
{

    Ссылок: 20
    public class Service
    {
        Ссылок: 2
        public void AddPartner(ApplicationContext _context, Partner partner)
        {
            _context.Partners.Add(partner);
            _context.SaveChanges();
        }
        Ссылок: 2
        public void DeletePartner(ApplicationContext _context, Partner partner)
        {
            _context.PartnerProducts.RemoveRange(_context.PartnerProducts.Where(p => p.Partner == partner.Id));
            _context.Partners.Remove(partner);
            _context.SaveChanges();
        }
        Ссылок: 2
        public void UpdatePartner(ApplicationContext _context, Partner partner)
        {
            _context.Partners.Update(partner);
            _context.SaveChanges();
        }
        Ссылок: 1
        public void SaveChanges(ApplicationContext _context)
        {
            _context.SaveChanges();
        }
        Ссылок: 2
        public List<PartnerView> GetAllPartnerInfo(ApplicationContext _context)
        {
            return _context.Partners
                .Select(p => new PartnerView
                {
                    Id = p.Id,
                    PartnerAddress = p.PartnerAddress,
                    PartnerName = p.PartnerName,
                    PartnerType = p.PartnerType,
                    PartnerDirector = p.PartnerDirector,
                    PartnerNumber = p.PartnerNumber,
                    PartnerEmail = p.PartnerEmail,
                    PartnerInn = p.PartnerInn,
                    PartnerRating = p.PartnerRating,
                    PartnerTypeName = p.PartnerType + " | " + p.PartnerName,
                    PartnerDiscount = GetDiscount(_context.PartnerProducts.Where(q => q.Partner == p.Id).Sum(q => q.PartnerCount) ?? 0)
                }).ToList();

        }
```

Рисунок 6 – Часть программного сервиса «PartnerService»

После определения и реализации архитектуры программного обеспечения осуществляется тестирование и разработка пользовательского интерфейса.

Процесс разработки интерфейса является комплексным и включает три основных этапа:

1. Проектирование – определение внешнего вида будущего приложения, выбор элементов интерфейса и установление их функционального назначения;

2. Прототипирование – создание интерактивного прототипа, его анализ, тестирование и внесение необходимых корректировок;

3. Стилизация – применение выбранного визуального стиля (цветовая гамма, шрифты, оформление элементов) к утверждённому прототипу.

В ходе проектирования было определено, что приложение будет состоять из двух основных компонентов:

1. Главного окна – основного интерфейса, предоставляющего доступ к ключевым данным и функциям программы;

2. Диалогового окна – окна для выполнения конкретных операций внутри приложения.

Главное окно должно включать в себя:

˜ Меню для навигации и доступа к основным функциям;

˜ Список партнёров, в котором отображается полная информация о каждом партнёре. Дополнительно реализована функция отображения скидки партнёра, рассчитываемой в зависимости от общего объёма продаж;

˜ Таблицу с историей продаж, содержащую сведения о наименовании продукции, количестве проданной продукции и датах реализации.

Диалоговое окно выполняет роль контейнера, загружающего определённые страницы или формы в зависимости от выполняемой операции. Каждая из этих страниц содержит элементы, соответствующие функциональным требованиям:

˜ Текстовые поля для ввода строковых или числовых данных;

˜ Выпадающие списки для выбора данных из заранее определённых вариантов;

˜ Календарь, используемый при работе с историей продаж;

˜ Кнопку выполнения операции, запускающую обработку введённых данных.

Таким образом, структура интерфейса обеспечивает удобство работы пользователя, позволяет централизованно управлять данными и выполнять необходимые операции с минимальным количеством действий.

Разрабатываемое пользовательское приложение создается с использованием технологии **WPF**, которая обеспечивает возможности построения современного и удобного графического интерфейса. Применение данной технологии позволяет реализовать все необходимые элементы управления и обеспечить интуитивно понятное взаимодействие пользователя с программой.

После формирования прототипа интерфейса выполняется его тщательная доработка. На этом этапе осуществляется корректировка внешнего вида компонентов, уточнение размеров элементов и шрифтов, а также оптимизация расположения элементов внутри окон приложения. Это необходимо для того, чтобы информация оставалась полностью читаемой и доступной при изменении размеров окон, а интерфейс сохранял удобство и логическую структуру отображения данных.

В результате проведенной работы формируются окончательные формы приложения, обеспечивающие полноценное взаимодействие пользователя с функционалом программы. Конечный вид интерфейса, включая расположение основных элементов и визуальное оформление окон, представлен на рисунках 7 и 8, что наглядно демонстрирует итоговую структуру пользовательского интерфейса и его готовность к эксплуатации.

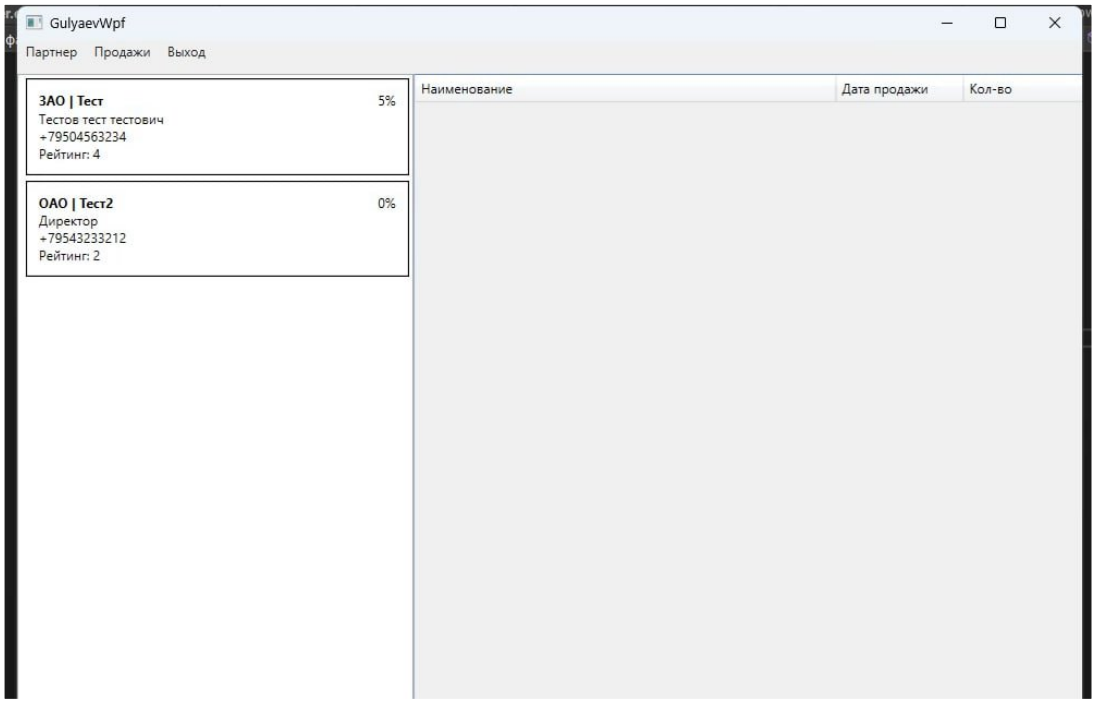

Рисунок 7 – Главное окно приложения

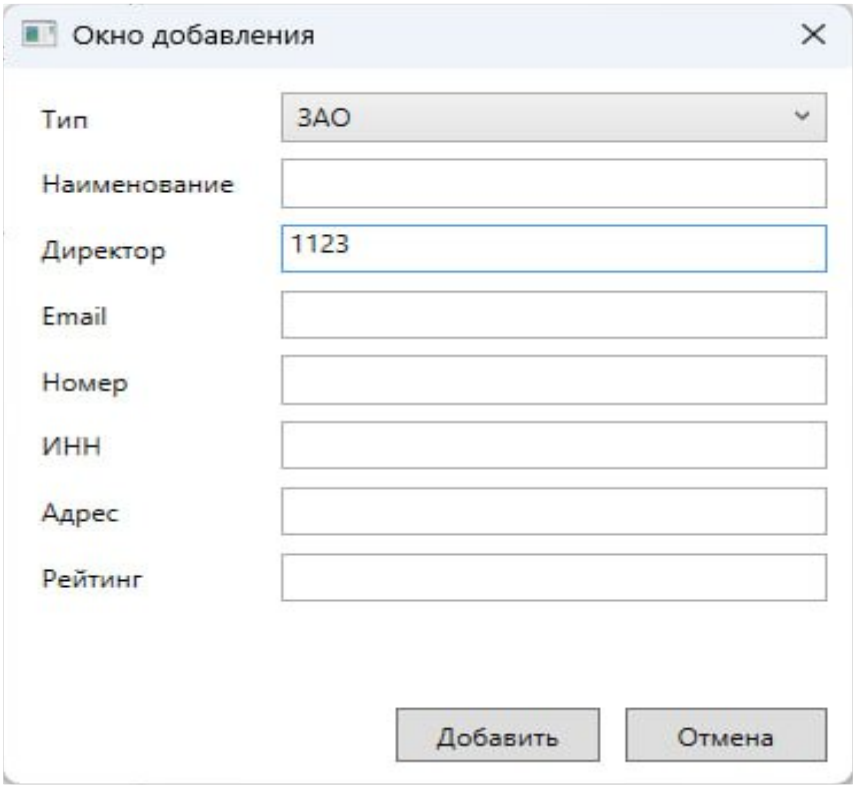

Рисунок 8 – Страница добавления партнера

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

Для обеспечения единообразия написания программного кода и поддержания его соответствия функциональным требованиям и установленным стандартам качества применяются различные методы. Эти методы можно разделить на программные и ручные.

К программным методам относятся встроенные в среды разработки анализаторы кода, а также сторонние расширения и плагины, которые расширяют возможности этих инструментов. В современном подходе к разработке также используются сервисы с элементами искусственного интеллекта, способные выполнять полный анализ кода, предоставлять рекомендации по его улучшению и даже формулировать предложения по рефакторингу.

К ручным методам относятся командный рефакторинг, проведение анализа кода, использование методологий разработки, предполагающих регулярные проверки и контроль качества на каждом этапе работы над

программным обеспечением, а также обсуждение и сравнение реализованных функций с исходными требованиями к проекту.

В рамках данной работы применяется сочетание программных и ручных методов контроля качества кода. В частности, используются встроенные анализаторы кода среды разработки, а также проводится обсуждение и сравнение реализованного функционала с заранее определёнными требованиями, что позволяет обеспечить соответствие программы заданным стандартам и требованиям.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Процесс инспектирования компонентов программного обеспечения на соответствие стандартам кодирования представляет собой комплексную процедуру, направленную на выявление отклонений программного кода от требований стиля, структуры и качества. Эти требования, как правило, определяются официальной документацией для каждого конкретного языка программирования. Основная цель инспекции заключается в обеспечении единообразия кода, его читаемости, поддерживаемости, оптимизации, а также снижении вероятности возникновения ошибок в процессе эксплуатации программного продукта.

В процессе инспектирования обычно участвуют три роли:

1. Автор кода – разработчик, создавший программный модуль;
2. Инспектор – специалист, проводящий анализ кода и вносящий необходимые изменения;
3. Наблюдатель – эксперт, привлекаемый для объективной оценки качества кода.

Выделяют несколько видов инспекций:

1. Формальная инспекция – инспекторы детально проверяют весь код, выявляют недочеты и оформляют вопросы автору. При подтверждении проблем разрабатываются планы по их устранению, а результаты фиксируются в отчёте;

2. Неформальная инспекция – проводится без строгих правил и сроков, инициируется и выполняется самим автором кода;

3. Чтение кода – код анализируется независимым разработчиком без участия автора;

4. Парное программирование – одновременный анализ и написание кода несколькими разработчиками.

Актуальные стандарты кодирования определяют требования к следующим аспектам:

- использование отступов;
- правила наименования переменных, объектов, файлов и других элементов;
- максимальная длина строк кода.

В рамках данной работы программный модуль разрабатывается на языке C#, для которого корпорация Microsoft разработала общие соглашения кодирования. Дополнительно используются рекомендации .NET Framework Guidelines, предназначенные для платформы .NET.

Среди инструментов, применяемых для инспектирования кода, можно выделить:

- встроенные средства среды разработки (например, Visual Studio или PyCharm);
- специализированные системы, такие как CodeCollaborator, Reviewable, RhodeCode и др.

Современные методы инспектирования также включают использование чат-ботов с искусственным интеллектом, встроенных в среды разработки. Однако данный подход не является полностью надежным, поскольку ИИ может допускать ошибки.

В данной работе инспектирование кода осуществлялось следующим образом:

1. Форма инспекции – неформальная, выполняемая самостоятельно;
2. Стандарт кодирования – .NET Framework Guidelines и соглашения от Microsoft;
3. Инструмент – встроенные средства среды разработки Visual Studio.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств

Отладка программного обеспечения является важным этапом процесса разработки, направленным на выявление и исправление ошибок в работе приложения. В ходе отладки проверяются различные сценарии использования программы, анализируются возможные варианты взаимодействия пользователя с функционалом, что позволяет своевременно обнаружить несоответствия и устранить их. Данный этап обычно проводится после тестирования основных компонентов программного продукта и применяется преимущественно к готовой версии пользовательского приложения, обеспечивая его корректную и стабильную работу.

В данной работе для полной отладки программы были составлены следующие сценарии:

1. Добавление продукции (дополнительно с вызовом отслеживаемых ошибок);
2. Изменение продукции (дополнительно с вызовом отслеживаемых ошибок);
3. Удаление продукции (дополнительно с вызовом отслеживаемых ошибок);
4. Добавление типа партнера (дополнительно с вызовом отслеживаемых ошибок);
5. Изменение типа партнера (дополнительно с вызовом отслеживаемых ошибок);
6. Удаление типа партнера (дополнительно с вызовом отслеживаемых ошибок);

7. Добавление партнера (дополнительно с вызовом отслеживаемых ошибок);

8. Изменение партнера (дополнительно с вызовом отслеживаемых ошибок);

9. Удаление партнера (дополнительно с вызовом отслеживаемых ошибок);

10. Добавление истории реализации продукции (дополнительно с вызовом отслеживаемых ошибок);

11. Изменение истории реализации продукции (дополнительно с вызовом отслеживаемых ошибок);

12. Удаление истории реализации продукции (дополнительно с вызовом отслеживаемых ошибок).

После прохождения всех этапов отладки готовый продукт вместе с документацией фиксируются в Git и загружаются на GitHub.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

В рамках данной работы осуществляется только модульное тестирование, так как данная разработанная программа не является модулем другой системы или не обладает функционалом взаимодействия с другими программными продуктами. По данной также причине проверяется только корректность выполнения разработанного функционала.

Целью данного тестирования является проверка корректного выполнения методов программного модуля, а задачами – проверка корректного выполнения функций создания, изменения, удаления предоставляемых объектов и их получение в различных условиях.

Пример части модульных тестов, составленные в данной работе, представлены на рисунке 9.

```
using Microsoft.EntityFrameworkCore;
using GulyaevLib;

namespace GulyaevTests
{
    [TestClass]
    [DoNotParallelize]
    Ссылок: 0
    public sealed class GulyaevServiceTests
    {
        Ссылок: 11
        private static ApplicationContext CreateContext()
        {
            return new ApplicationContext();
        }

        [TestMethod]
        Ссылок: 0
        public void AddPartner_AddsPartnerToDatabase()
        {
            using var context = CreateContext();
            using var transaction = context.Database.BeginTransaction();
            var service = new Service();
            var partner = new Partner
            {
                PartnerName = "Test Partner",
                PartnerType = "TP",
                PartnerDirector = "Director",
                PartnerEmail = "test@example.com",
                PartnerNumber = "1234567890",
                PartnerAddress = "Test Address",
                PartnerInn = "1234567890",
                PartnerRating = 1
            };

            service.AddPartner(context, partner);

            var exists = context.Partners.Any(p => p.Id == partner.Id);
            Assert.IsTrue(exists);
            transaction.Rollback();
        }

        [TestMethod]
        Ссылок: 0
        public void DeletePartner_RemovesPartnerFromDatabase()
        {
            using var context = CreateContext();
            using var transaction = context.Database.BeginTransaction();
            var service = new Service();
            var partner = new Partner
            {
                PartnerName = "Partner To Delete",
                PartnerType = "PD"
```

Рисунок 9 – Пример модульных тестов

3.1.7. Интегрирование программных модулей

Разрабатываемая данная система предназначена для управления партнёрами и учёта реализации данной продукции. Архитектура приложения построена по многоуровневому принципу, при котором каждый уровень выполняет определённые функции и также отвечает за отдельную часть работы программы.

Первый уровень включает модели данных, которые описывают основные сущности системы, такие как партнёры, продукция и операции реализации. Данные модели определяют структуру хранимой информации и правила её использования в приложении.

Второй уровень представляет собой механизм взаимодействия с базой данных, обеспечивающий сохранение, изменение и получение информации. Он отвечает за корректную работу с данными и их целостность.

Третий уровень содержит прикладную логику системы. Здесь реализуются основные алгоритмы обработки данных, выполняются расчёты, проверка корректности информации и подготовка данных для отображения пользователю.

Четвёртый уровень представляет пользовательский интерфейс, который обеспечивает взаимодействие пользователя с программой. С его помощью выполняется ввод данных, просмотр информации и выполнение необходимых операций.

Разделение программы на несколько уровней позволяет повысить удобство разработки, упростить сопровождение программного продукта и облегчить его дальнейшую модернизацию.

В основе реализации системы используются технологии .NET и Entity Framework, которые обеспечивают работу приложения и взаимодействие с базой данных.

3.1.8. Верификация и аттестация программного обеспечения

Верификация и аттестация программного обеспечения являются важными этапами оценки качества разработанного программного продукта. Эти процессы также направлены на подтверждение соответствия программного обеспечения установленным требованиям, стандартам и нормативной документации.

Верификация представляет собой процесс проверки того, что разработанное программное обеспечение корректно реализует поставленные требования и соответствует разработанным спецификациям. Основная цель данного этапа заключается в выявлении возможных несоответствий между требованиями к системе и фактически реализованным функционалом.

В рамках данной работы для проведения верификации были использованы следующие методы:

1. Анализ соответствия программного продукта установленным требованиям;

2. Анализ архитектуры приложения и структуры программных модулей;

3. Анализ программного кода с проведением тестирования.

В результате выполнения указанных мероприятий было установлено, что разработанный программный продукт соответствует поставленным требованиям. Архитектура приложения обеспечивает логичное разделение функциональности между программными модулями и также способствует повышению читаемости и поддерживаемости кода. Кроме того, программный код соответствует установленным правилам разработки и обладает предсказуемой логикой работы.

Аттестация представляет собой процесс подтверждения соответствия программного обеспечения установленным стандартам, нормативным требованиям и критериям качества. Проведение аттестации позволяет оценить надежность, устойчивость и функциональные возможности программного продукта.

Процедура аттестации может осуществляться на основе различных государственных и международных стандартов, в том числе ГОСТ Р ИСО/МЭК 25000-2015, а также других нормативных документов, регламентирующих требования к качеству программного обеспечения.

Для проведения аттестации были применены следующие методы:

1. Функциональное тестирование — проверка соответствия программного продукта заявленным функциональным требованиям посредством выполнения тестовых сценариев;

2. Нагрузочное и стресс-тестирование — оценка производительности системы и её устойчивости при повышенных нагрузках;

3. Интеграционное тестирование — проверка корректности взаимодействия различных модулей и компонентов системы после их

объединения. В данном случае осуществлялась проверка взаимодействия оконного приложения с программными библиотеками.

По результатам проведённых мероприятий разработанный программный продукт продемонстрировал положительные результаты проверки и подтвердил соответствие установленным требованиям и также критериям качества.

Диаграмма классов отражает статическую структуру программной системы: основные классы проекта, их атрибуты (свойства) и взаимосвязи между ними. На ней представлены классы-сущности, соответствующие таблицам базы данных (Partner, ProductType, Product, PartnerProduct), класс контекста (ApplicationContext), а также сервисный класс (Service), содержащий бизнес-логику приложения. Связи между классами показывают направления навигации (один ко многим, многие к одному) и отражают архитектуру, реализованную с использованием Entity Framework Core. Диаграмма даёт наглядное представление о ключевых компонентах программного модуля и служит основой для понимания его устройства и дальнейшего развития.

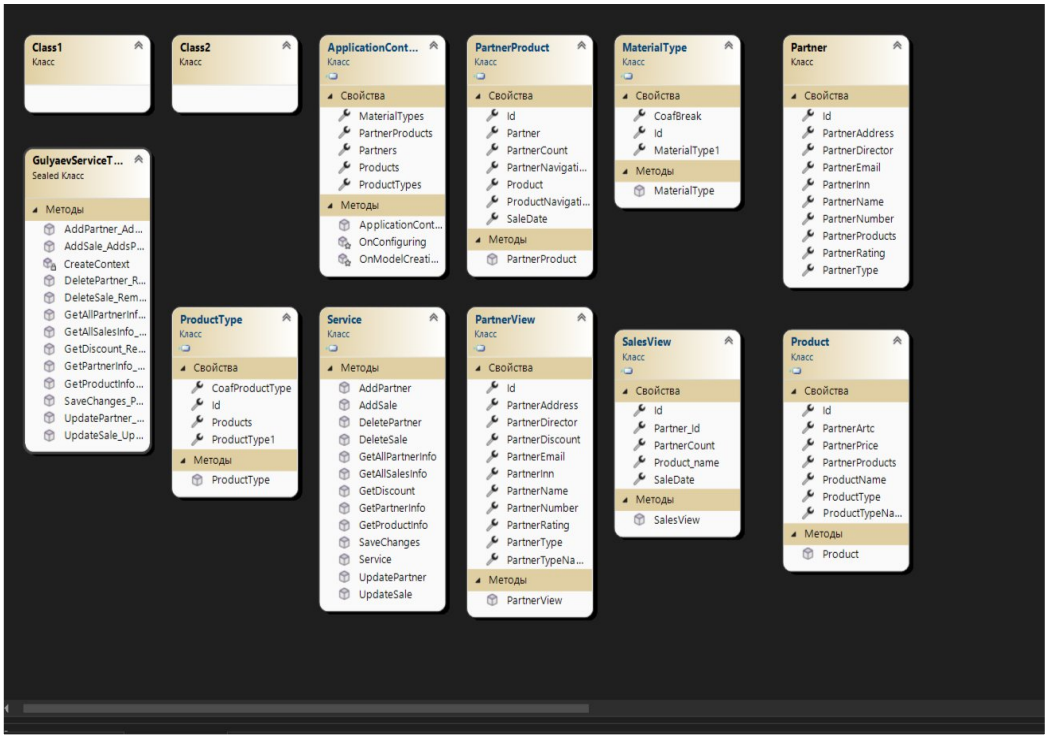

Рисунок 10 – Диаграмма классов

ЗАКЛЮЧЕНИЕ

В ходе прохождения производственной практики в период с 09 по 17 марта 2026 года были закреплены ранее полученные теоретические знания, а также расширены и также углублены профессиональные навыки в области разработки программного обеспечения.

В процессе выполнения практических заданий были достигнуты следующие результаты:

1. Получен практический опыт анализа предметной области разрабатываемой системы;
2. Сформированы требования к программному продукту на основе проведённого анализа;
3. Выполнена разработка программных модулей в соответствии с установленными требованиями;
4. Проведено тестирование разработанных программных модулей;
5. Выполнена отладка программных компонентов с использованием специализированных программных средств;
6. Проведён рефакторинг и оптимизация программного кода;
7. Осуществлено инспектирование модулей программного обеспечения на соответствие стандартам кодирования;
8. Проведены процедуры верификации и аттестации разработанного программного обеспечения.

На основании выполненных задач можно сделать вывод о том, что поставленные цели производственной практики были успешно достигнуты.

В результате прохождения практики был получен практический опыт интеграции программных модулей в состав программного продукта, выполнения отладки программных компонентов с применением специализированных средств разработки, создания тестовых наборов и также сценариев тестирования, а также проведения инспектирования программного кода на соответствие установленным стандартам разработки.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Федорова, Г. Н. Участие в интеграции программных модулей: учебное пособие / Г. Н. Федорова. — Москва: Издательский центр «Академия», 2016. — 304 с.

2. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: учебное пособие / Г. Н. Федорова. — Москва: Курс: ИНФРА-М, 2016. — 336 с.

3. Черников, Б. В. Оценка качества программного обеспечения: практикум: учебное пособие / Б. В. Черников, Б. Е. Поклонов; под ред. Б. В. Черникова. — Москва: ФОРУМ: ИНФРА-М, 2012. — 400 с.

4. Исаченко, О. В. Программное обеспечение компьютерных сетей: учебное пособие / О. В. Исаченко. — Москва: ИНФРА-М, 2013. — 117 с.

5. Ананьева, Т. Н. Стандартизация, сертификация и управление качеством программного обеспечения: учебное пособие / Т. Н. Ананьева, Н. Г. Новикова, Г. Н. Исаев. — Москва: ИНФРА-М, 2016. — 232 с.

6. Гагарина, Л. Г. Технология разработки программного обеспечения: учебное пособие / Л. Г. Гагарина, Б. Д. Виснадул, Е. В. Кокорева. — Москва: ИНФРА-М, 2013. — 400 с.

7. Гагарина, Л. Г. Разработка и эксплуатация автоматизированных информационных систем: учебное пособие / Л. Г. Гагарина. — Москва: ФОРУМ: ИНФРА-М, 2013. — 384 с.

8. Рудаков, А. В. Технология разработки программных продуктов: учебное пособие / А. В. Рудаков. — Москва: Академия, 2010. — 208 с.

9. Черников, Б. В. Управление качеством программного обеспечения: учебник / Б. В. Черников. — Москва: ФОРУМ : ИНФРА-М, 2012. — 240 с.

Интернет-источники

10. Единая Россия. Официальный сайт политической партии. — URL: https://er.ru (дата обращения: 14.03.2026).

11. Региональное отделение Единая Россия в Пермский край. Официальный сайт. — URL: https://perm.er.ru (дата обращения: 14.03.2026).

12. Портал органов власти Пермский край. — URL: https://permkrai.ru (дата обращения: 14.03.2026).

13. Документация по платформе .NET. — URL: https://learn.microsoft.com/ru-ru/dotnet/ (дата обращения: 16.03.2026).

14. Руководство по работе с Entity Framework. — URL: https://learn.microsoft.com/ru-ru/ef/ (дата обращения: 14.03.2026).

ПРИЛОЖЕНИЯ

Приложение 1

Требования к программному обеспечению

Разрабатываемое программное обеспечение предназначено для автоматизации процессов обработки, хранения и поиска информации, а также для повышения эффективности работы пользователей при выполнении служебных задач. Система должна обеспечивать удобный доступ к необходимым данным, их структурированное хранение и возможность последующей обработки.

К основным требованиям, предъявляемым к программному обеспечению, относятся: надежность функционирования системы, удобство и понятность пользовательского интерфейса, безопасность хранения данных, а также возможность взаимодействия с другими информационными системами.

Функциональные требования к программному обеспечению включают:

- ввод, редактирование и удаление данных;
- хранение информации в структурированной базе данных;
- поиск и просмотр информации по заданным параметрам;
- обработку и систематизацию данных;
- формирование отчетной информации;
- контроль целостности и корректности данных.

Программное обеспечение должно обеспечивать стабильную работу на персональных компьютерах под управлением операционной системы семейства Windows. Пользовательский интерфейс системы должен быть интуитивно понятным и обеспечивать удобную навигацию по основным разделам приложения.

Система должна обеспечивать защиту информации от несанкционированного доступа, а также поддерживать возможность резервного копирования и восстановления данных при возникновении программных или аппаратных сбоев. Кроме того, программное обеспечение

должно обеспечивать корректную работу с базой данных и предотвращать потерю или повреждение информации.

Также важным требованием является масштабируемость программного обеспечения. Система должна поддерживать возможность обновления и расширения функциональных возможностей, что позволит адаптировать её к изменяющимся требованиям и условиям эксплуатации.